

Advanced Dynare

Chapter XI

Perfect foresight simulations

Johannes Pfeifer

University of the Bundeswehr Munich

October 2025, Bank of Korea



Copyright © 2015-2025 Dynare Team
License: [Creative Commons Attribution-ShareAlike 4.0](https://creativecommons.org/licenses/by-sa/4.0/)

Introduction

- Perfect foresight = agents perfectly anticipate all future shocks
- Concretely, at period 1:
 - agents learn the value of all future shocks;
 - with shared knowledge of model and future shocks, agents can compute their optimal plans for all future periods;
 - optimal plans do not need to be adjusted in later periods
⇒ model behaves as if it were deterministic
- Cost of this approach: the effect of future uncertainty is not taken into account (e.g. no precautionary motive)
- Advantage: numerical solution can be computed exactly (up to rounding errors), contrarily to perturbation or many global solution methods for rational expectations models
- In particular, nonlinearities fully taken into account (e.g. occasionally binding constraints)

Outline

1. Presentation of the problem
2. Solution techniques
3. Shocks: temporary/permanent, unexpected/pre-announced
4. Recent Improvements
5. Perfect foresight with expectation errors
6. Extended path

Outline

1. Presentation of the problem
2. Solution techniques
3. Shocks: temporary/permanent, unexpected/pre-announced
4. Recent Improvements
5. Perfect foresight with expectation errors
6. Extended path

The (deterministic) neoclassical growth model

- Take the consumption choice of a household knowing TFP $\{A_t\}_{t=1}^{\infty}$ and taking k_0 as given:

$$\max_{\{c_t\}_{t=1}^{\infty}} \sum_{t=1}^{\infty} \beta^{t-1} \frac{c_t^{1-\sigma} - 1}{1-\sigma} \quad (1)$$

$$s.t. \ c_t + k_t = A_t k_{t-1}^{\alpha} + (1-\delta)k_{t-1} \quad (2)$$

- First order conditions:

$$c_t^{-\sigma} = \beta c_{t+1}^{-\sigma} \left(\alpha A_{t+1} k_t^{\alpha-1} + 1 - \delta \right) \quad (3)$$

$$c_t + k_t = A_t k_{t-1}^{\alpha} + (1-\delta)k_{t-1} \quad (4)$$

- Note the absence of stochastic elements
→ no expectations, no probability distribution

The (deterministic) neoclassical growth model: simulation scenario

- Assume household wakes up in $t = 1$ with capital k_0 at its steady state
- Steady state conditional on A being constant is given by:

$$k = \left[\frac{1 - \beta(1 - \delta)}{\beta\alpha A} \right]^{\frac{1}{\alpha-1}} \quad (5)$$

$$c = Ak^\alpha - \delta k \quad (6)$$

- In $t = 1$ the household is informed that TFP will be 20% higher in $t = 1$ and back to $A = 1$ ever after
- What is the household's consumption response?

Dynare code (1/3)

rcb_basic.mod

```
var c k;  
varexo A;  
parameters alpha beta gamma delta;  
  
alpha=0.5;  
beta=0.95;  
gamma=0.5;  
delta=0.02;  
  
model;  
    c + k = A*k(-1)^alpha + (1-delta)*k(-1);  
    c^(-gamma) = beta*c(+1)^(-gamma)*(alpha*A(+1)*k^(alpha-1) + 1 - delta);  
end;
```

Dynare code (2/3)

rcb_basic.mod

```
steady_state_model;  
    k = ((1-beta*(1-delta))/(beta*alpha*A))^(1/(alpha-1));  
    c = A*k^alpha-delta*k;  
end;  
  
// set initial and terminal condition to steady state  
initval;  
    A = 1;  
end;  
steady;
```


Dynare code (3/3)

rcb_basic.mod

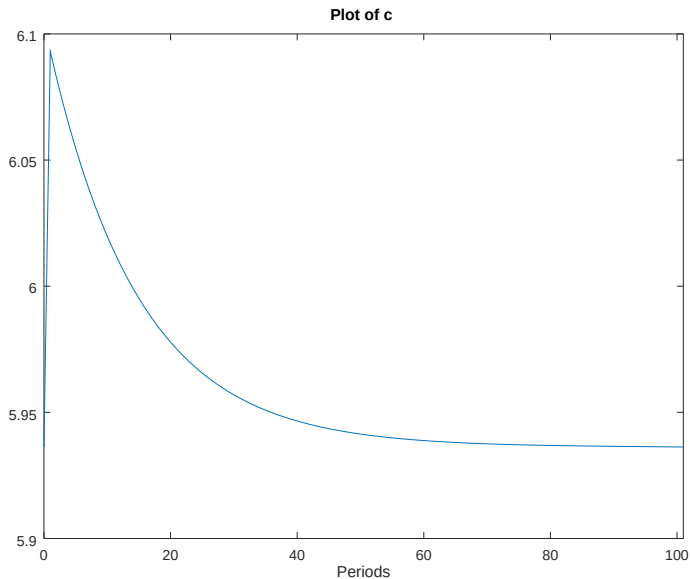
```
// Declare a positive technological shock in period 1
shocks;
    var A;
    periods 1;
    values 1.2;
end;

// Prepare the deterministic simulation over 100 periods
perfect_foresight_setup(periods=100);

// Perform the simulation
perfect_foresight_solver;

// Display the path of consumption
rplot c;
```

Simulated consumption path



The general problem

- Perfect foresight problem can be represented as

$$f(y_{t+1}, y_t, y_{t-1}, u_t) = 0, \quad (7)$$

with

- y : vector of endogenous variables
 - u : vector of exogenous variables/shocks
- Model requires a many endogenous variables y as equations in f

Return to the neoclassical growth model

- The neoclassical growth model has the representation:

$$y_t = \begin{pmatrix} c_t \\ k_t \end{pmatrix} \quad (8)$$

$$u_t = A_t \quad (9)$$

$$f(y_{t+1}, y_t, y_{t-1}, u_t) = \begin{bmatrix} c_t^{-\sigma} - \beta c_{t+1}^{-\sigma} (\alpha A_{t+1} k_t^{\alpha-1} + 1 - \delta) \\ c_t + k_t - A_t k_{t-1}^{\alpha} + (1 - \delta) k_{t-1} \end{bmatrix} = 0 \quad (10)$$

Dealing with higher order leads and lags

- Models can always be transformed to canonical form with at most one lead and lag using *auxiliary* variables
→ transformation done automatically by Dynare
- Take variable with two leads x_{t+2} :
 - create a new auxiliary variable a
 - add a new equation: $a_t = x_{t+1}$
 - replace all occurrences of x_{t+2} by a_{t+1}
- Symmetric process for variables with more than one lag
- With future uncertainty, transformation is more elaborate (but still possible) on variables with leads
- `write_latex_original_model` and `write_latex_dynamic_model` show model before and after substitution

Steady state

- A steady state y for the model satisfies

$$f(y, y, y, u) = 0 \quad (11)$$

- Note that, in this context, a steady state is conditional on
 - steady state values of exogenous variables u
 - structural parameters (implicit in f)
- Careful: such a steady state may not be unique
- The steady state is computed/checked by Dynare with the `steady` command
→ triggers a nonlinear solver if no analytical steady state is provided

Towards a solution

- In each period, agent needs to choose consumption today, taking into account
 - the known time path of the exogenous states
 - the current endogenous states that cannot be altered anymore (capital)
 - the consequences of today's actions for future decisions
 - Assume finite horizon problem that terminates in period T with
 - initial states in y_0 given
 - terminal choice y_{T+1} known
- problem boils down to simultaneous equation system with $n_y \times T$ equations in $n_y \times T$ unknowns

A two-boundary value problem

- Problem can be formulated as stacked system over T periods, given y_0 and y_{T+1}

$$\begin{cases} f(y_2, y_1, y_0, u_1) = 0 \\ f(y_3, y_2, y_1, u_2) = 0 \\ \vdots \\ f(y_{T+1}, y_T, y_{T-1}, u_T) = 0 \end{cases} \quad (12)$$

- More compactly:

$$F(Y) = 0, \quad (13)$$

where $Y = \begin{bmatrix} y'_1 & y'_2 & \dots & y'_T \end{bmatrix}'$ and “exogenous” elements $y_0, y_{T+1}, u_1 \dots u_T$ are implicit

- Stacked equation system can be solved using a Newton-type method

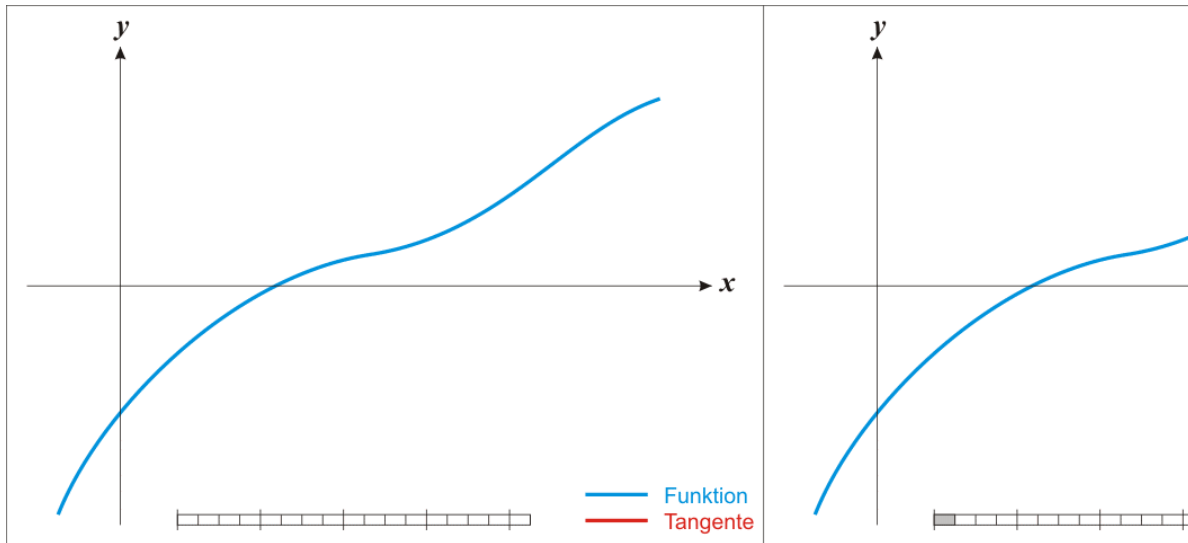
Approximating infinite-horizon problems

- What if you are interested in solving *infinite*-horizon problem instead of *finite*-horizon one?
- One option: compute recursive policy function (e.g. perturbation), but this is challenging
 - in general, this function is defined over infinite-dimensional space (all future shocks are state variables)
 - in case of a return to equilibrium, state-space is finite (starting from date where all shocks are zero), but a projection method would still be needed → Dynare does not support this
- Often easier way in case of return to equilibrium: approximate solution by a finite-horizon problem
→ computing the trajectory with $y_{T+1} = \bar{y}$ and T large enough
- Drawback compared to policy function approach: solution specific to a given sequence of shocks, and not generic

Outline

1. Presentation of the problem
2. Solution techniques
3. Shocks: temporary/permanent, unexpected/pre-announced
4. Recent Improvements
5. Perfect foresight with expectation errors
6. Extended path

The Newton method (unidimensional)



Copyright © 2005 Ralf Pfeifer / [Creative Commons Attribution-ShareAlike 3.0](#)

The Newton method (multidimensional)

- Start from an initial guess $Y^{(0)}$
- Iterate with updated solutions $Y^{(k+1)}$ obtained by solving linear system:

$$F(Y^{(k)}) + \left[\frac{\partial F}{\partial Y} \right] (Y^{(k+1)} - Y^{(k)}) = 0 \quad (14)$$

- Terminate if problem solved or no more improvement:

$$\|F(Y^{(k)})\|_{\infty} < \varepsilon_F \quad \text{or} \quad \|Y^{(k+1)} - Y^{(k)}\|_{\infty} < \varepsilon_Y \quad (15)$$

- Convergence may never happen if function is ill-behaved or initial guess $Y^{(0)}$ too far from a solution
→ to avoid an infinite loop, abort after a given number of iterations

Controlling the Newton algorithm from Dynare

The following options to the `perfect_foresight_solver` can be used to control the Newton algorithm:

`maxit` Maximum number of iterations before aborting (default: 50)

`tolf` Convergence criterion based on function value (ε_F) (default: 10^{-5})

`tolx` Convergence criterion based on change in the function argument (ε_Y) (default: 10^{-5})

`stack_solve_algo` selects between the different flavors of Newton algorithms (more later)

A practical difficulty

- Jacobian matrix of dimension $n_y T \times n_y T$ can be really large!
- Three alternative ways of dealing with large problem size:
 1. Exploit particular structure of Jacobian using a technique developed by Laffargue (1990), Boucekkine (1995) and Juillard (1996) (was the default method in Dynare ≤ 4.2)
 2. Handle the Jacobian as one large sparse matrix (now the default method)
 3. Dulmage/Mendelsohn block decomposition, which is a divide-and-conquer method (can be combined with one of the previous two methods)
→ will be the default in Dynare 7

Shape of the Jacobian

$$\frac{\partial F}{\partial Y} = \begin{pmatrix} B_1 & C_1 & & & & \\ A_2 & B_2 & C_2 & & & \\ & \ddots & \ddots & \ddots & & \\ & & A_t & B_t & C_t & \\ & & & \ddots & \ddots & \ddots \\ & & & & A_{T-1} & B_{T-1} & C_{T-1} \\ & & & & & A_T & B_T \end{pmatrix} \quad (16)$$

$$A_s = \frac{\partial f}{\partial y_{t-1}}(y_{s+1}, y_s, y_{s-1}) \quad (17)$$

$$B_s = \frac{\partial f}{\partial y_t}(y_{s+1}, y_s, y_{s-1}) \quad (18)$$

$$C_s = \frac{\partial f}{\partial y_{t+1}}(y_{s+1}, y_s, y_{s-1}) \quad (19)$$

Laffargue-Boucekkine-Juillard algorithm (1/5)

- The idea is to triangularize the stacked system:

$$\begin{pmatrix} B_1 & C_1 & & & \\ A_2 & B_2 & C_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \ddots & \ddots & \\ & & & A_{T-1} & B_{T-1} & C_{T-1} \\ & & & & A_T & B_T \end{pmatrix} \Delta Y = - \begin{pmatrix} f(y_2, y_1, y_0, u_1) \\ f(y_3, y_2, y_1, u_2) \\ \vdots \\ \vdots \\ f(y_T, y_{T-1}, y_T, u_{T-1}) \\ f(y_{T+1}, y_T, y_{T-1}, u_T) \end{pmatrix}, \quad (20)$$

eliminating elements below the main diagonal by proceeding recursively from the top

Laffargue-Boucekkine-Juillard algorithm (2/5)

- The first period is special due to the initial condition:

$$\begin{pmatrix} I & D_1 & & & & \\ & B_2 - A_2 D_1 & C_2 & & & \\ & A_3 & B_3 & C_3 & & \\ & & \ddots & \ddots & \ddots & \\ & & & A_{T-1} & B_{T-1} & C_{T-1} \\ & & & & A_T & B_T \end{pmatrix} \Delta Y = - \begin{pmatrix} d_1 \\ f(y_3, y_2, y_1, u_2) + A_2 d_1 \\ f(y_4, y_3, y_2, u_3) \\ \vdots \\ f(y_T, y_{T-1}, y_T, u_{T-1}) \\ f(y_{T+1}, y_T, y_{T-1}, u_T) \end{pmatrix}, \quad (21)$$

where

- $D_1 = B_1^{-1} C_1$
- $d_1 = B_1^{-1} f(y_2, y_1, y_0, u_1)$

Laffargue-Boucekkine-Juillard algorithm (3/5)

- Normal iteration of the triangularization:

$$\begin{pmatrix} I & D_1 & & & & \\ & I & D_2 & & & \\ & & B_3 - A_3 D_2 & C_3 & & \\ & & & \ddots & \ddots & \\ & & & & A_{T-1} & B_{T-1} & C_{T-1} \\ & & & & & A_T & B_T \end{pmatrix} \Delta Y = - \begin{pmatrix} d_1 \\ d_2 \\ f(y_4, y_3, y_2, u_3) + A_3 d_2 \\ \vdots \\ f(y_T, y_{T-1}, y_T, u_{T-1}) \\ f(y_{T+1}, y_T, y_{T-1}, u_T) \end{pmatrix}, \quad (22)$$

where

- $D_2 = (B_2 - A_2 D_1)^{-1} C_2$
- $d_2 = (B_2 - A_2 D_1)^{-1} (f(y_3, y_2, y_1, u_2) + A_2 d_1)$

Laffargue-Boucekkine-Juillard algorithm (4/5)

- Final iteration:

$$\begin{pmatrix} I & D_1 & & & & \\ & I & D_2 & & & \\ & & I & D_3 & & \\ & & & \ddots & \ddots & \\ & & & & I & D_{T-1} \\ & & & & & I \end{pmatrix} \Delta Y = - \begin{pmatrix} d_1 \\ d_2 \\ d_3 \\ \vdots \\ d_{T-1} \\ d_T \end{pmatrix} \quad (23)$$

where

$$d_T = (B_T - A_T D_{T-1})^{-1} (f(y_{T+1}, y_T, y_{T-1}, u_T) + A_T d_{T-1}) \quad (24)$$

Laffargue-Boucekkine-Juillard algorithm (5/5)

- The triangular system can then be solved by backward iteration:

$$y_T^{k+1} = y_T^k - d_T \quad (25)$$

$$y_{T-1}^{k+1} = y_{T-1}^k - d_{T-1} - D_{T-1}(y_T^{k+1} - y_T^k) \quad (26)$$

$$\vdots \quad (27)$$

$$y_1^{k+1} = y_1^k - d_1 - D_1(y_2^{k+1} - y_2^k) \quad (28)$$

- Advantage: no need to ever store the whole Jacobian, only the D_s and d_s
→ memory efficiency made it the default method in Dynare ≤ 4.2
- Still available as option `stack_solve_algo=6` of `perfect_foresight_solver` command (or `stack_solve_algo=1` in Dynare 6)

Sparse matrices (1/3)

- Consider the following matrix with most elements equal to zero:

$$A = \begin{pmatrix} 0 & 0 & 2.5 \\ -3 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \quad (29)$$

- Dense matrix storage (in column-major order) treats it as a one-dimensional array:

$$[0, -3, 0, 0, 0, 0, 2.5, 0, 0]$$

- Sparse matrix storage:

- views it as a list of triplets (i, j, v) where (i, j) is a matrix coordinate and v a non-zero value
- A would be stored as

$$\{(2, 1, -3), (1, 3, 2.5)\}$$

Sparse matrices (2/3): memory considerations

- In the general case, given an $m \times n$ matrix with k non-zero elements:
 - dense matrix storage = $8mn$ bytes
 - sparse matrix storage = $16k$ bytes
 - sparse storage more memory-efficient as soon as $k < mn/2$
(assuming 32-bit integers and 64-bit floating point numbers)
- In practice, sparse storage becomes interesting if $k \ll mn/2$, because linear algebra algorithms are vectorized

Sparse matrices (3/3): DSGE models

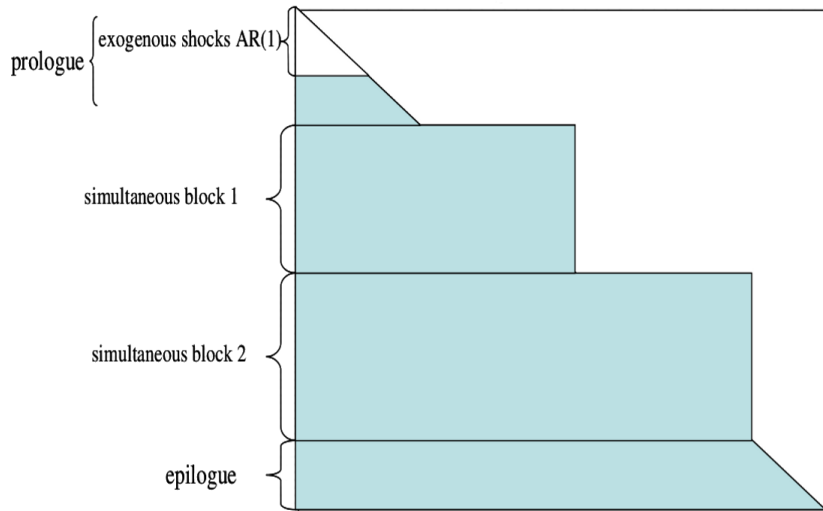
- The Jacobian of the deterministic problem is a sparse matrix:
 - Lots of zero blocks
 - The A_s , B_s and C_s usually are themselves sparse
- Many optimized algorithms for sparse matrices exist, including solver for linear equation solving in Newton algorithm
- Sparse matrices exist as native objects in MATLAB/Octave (see `sparse` command)
- Works well for medium-size deterministic models
- Nowadays more efficient than Laffargue-Boucekkine-Juillard, despite not exploiting the particular structure of the Jacobian
⇒ current default method in Dynare 6 (`stack_solve_algo=0`)

Block decomposition (1/3): idea

- Idea: apply a divide-and-conquer technique to model simulation
→ block triangularization of the Jacobian
- Principle: identify recursive as well as simultaneous blocks in the model structure
 1. First block (prologue): equations that only involve variables determined by previous equations
→ e.g. $AR(1)$ processes
 2. Last block (epilogue): pure output/reporting equations
 3. In between: simultaneous blocks that depend recursively on each other
- Identification of blocks and block types is performed through matching between variables and equations (normalization), then reordering both

Block decomposition (2/3)

Form of the reordered Jacobian (equations in lines, variables in columns)



Block decomposition (3/3)

- Can provide significant speed-up on large models
- Implemented in Dynare by Ferhat Mihoubi
- Available as option `block` to the `model` command
- Bigger gains when used in conjunction with `bytecode` or `use_dll` option
- Can be combined with any flavor of the Newton method applied to each individual block

Homotopy

- Another divide-and-conquer method, but in the shocks dimension
- Useful if large shocks prevent convergence
- Idea: achieve convergence for smaller shocks, use result as initial guess for bigger shock
- Algorithm:
 1. $\lambda \leftarrow 0$: scaling factor of shocks (simulation succeeds when $\lambda = 1$)
 2. $s \leftarrow 1$: step size
 3. Try to compute simulation with shock scaling factor equal to $\lambda + s$, using last successful computation as initial guess (with initial steady state at all t as initial guess)
 - If success: $\lambda \leftarrow (\lambda + s)$. Stop if $\lambda = 1$. Otherwise possibly increase s .
 - If failure: diminish s .
 4. Go to 3
- Works with both temporary and permanent shocks (*i.e.* shocks and endval)
- Can be combined with any deterministic solver
- Enabled by default in `perfect_foresight_solver` (disable with option `no_homotopy`)

Outline

1. Presentation of the problem
2. Solution techniques
3. Shocks: temporary/permanent, unexpected/pre-announced
4. Recent Improvements
5. Perfect foresight with expectation errors
6. Extended path

Example: neoclassical growth model with investment

The social planner problem is as follows:

$$\max_{\{c_t, \ell_t, k_t\}_{t=1}^{\infty}} \sum_{t=1}^{\infty} \beta^{t-1} u(c_t, \ell_t) \quad (30)$$

s.t.

$$y_t + (1 - \delta)k_{t-1} = c_t + k_t \quad (31)$$

$$y_t = A_t f(k_{t-1}, \ell_t) \quad (32)$$

$$A_t = A^* e^{a_t} \quad (33)$$

$$a_t = \rho a_{t-1} + \varepsilon_t, \quad (34)$$

where ε_t is an exogenous shock and k_0 given.

Functional forms

- Utility function is non-separable:

$$u(c_t, \ell_t) = \frac{\left(c_t^\theta (1 - \ell_t)^{1-\theta}\right)^{1-\tau}}{1 - \tau} \quad (35)$$

- CES production function:

$$f(k_{t-1}, \ell_t) = \left(\alpha k_{t-1}^\psi + (1 - \alpha) \ell_t^\psi\right)^{\frac{1}{\psi}} \quad (36)$$

First order conditions

- Euler equation:

$$u_c(c_t, \ell_t) = \beta \mathbb{E}_t \left[u_c(c_{t+1}, \ell_{t+1}) \left(A_{t+1} f_k(k_t, \ell_{t+1}) + 1 - \delta \right) \right] \quad (37)$$

- Consumption/leisure tradeoff:

$$\frac{u_\ell(c_t, \ell_t)}{u_c(c_t, \ell_t)} + A_t f_\ell(k_{t-1}, \ell_t) = 0 \quad (38)$$

- Resource constraint:

$$c_t + k_t = A_t f(k_{t-1}, \ell_t) + (1 - \delta)k_{t-1} \quad (39)$$

Dynare code (1/3)

rbc_det1.mod

```
var k, y, L, c, A, a;  
varexo epsilon;  
parameters beta, theta, tau, alpha, psi, delta, rho, Astar;  
  
beta = 0.99;  
theta = 0.357;  
tau = 2;  
alpha = 0.45;  
psi = -0.1;  
delta = 0.02;  
rho = 0.8;  
Astar = 1;
```


Dynare code (2/3)

rbc_det1.mod

```
model;
  a = rho*a(-1) + epsilon;
  A = Astar*exp(a);
  y = A*(alpha*k(-1)^psi+(1-alpha)*L^psi)^(1/psi);
  k = y-c + (1-delta)*k(-1);
  (1-theta)/theta*c/(1-L) - (1-alpha)*(y/L)^(1-psi);
  (c^theta*(1-L)^(1-theta))^(1-tau)/c =
    beta*(c(+1)^theta*(1-L(+1))^(1-theta))^(1-tau)/c(+1)
    *(alpha*(y(+1)/k)^(1-psi)+1-delta);
end;
```

Dynare code (3/3)

```
steady_state_model;
    a = epsilon/(1-rho);
    A = Astar*exp(a);
    y_over_k=((1/beta-1+delta)/alpha)^(1/(1-psi));
    c_over_k=y_over_k-delta;
    l_over_k=((y_over_k/A)^psi-alpha)/(1-alpha))^(1/psi);
    y_over_l=y_over_k/l_over_k;
    c_over_l=c_over_k/l_over_k;

    % Compute steady state of the endogenous variables.
    L=1/(1+c_over_l/((1-alpha)*theta/(1-theta)*y_over_l^(1-psi)));
    c=c_over_l*L;
    k=L/l_over_k;
    y=y_over_k*k;
end;
```

Scenario 1: Return to equilibrium

rbc_det1.mod

Return to equilibrium starting from $k_0 = 0.5\bar{k}$.

```
...
steady;

ik = varlist_indices('k',M_.endo_names);
kstar = oo_.steady_state(ik);

histval;
    k(0) = kstar/2;
end;

perfect_foresight_setup(periods=300);
perfect_foresight_solver;
```

Scenario 2: A temporary shock to TFP

rbc_det2.mod

- Economy starts from steady state
- At beginning of period 1: unexpected negative shock $\varepsilon_1 = -0.1$

```
...  
steady;  
  
shocks;  
  var epsilon;  
  periods 1;  
  values -0.1;  
end;  
  
perfect_foresight_setup(periods=300);  
perfect_foresight_solver;
```

Scenario 3: Pre-announced favorable shocks in the future

rbc_det3.mod

- Economy starts from steady state
- Sequence of positive shocks to A_t : 4% in period 5 and additional 1% during the 4 following periods

```
...
steady;

shocks;
  var epsilon;
  periods 4, 5:8;
  values 0.04, 0.01;
end;

perfect_foresight_setup(periods=300);
perfect_foresight_solver;
```

Scenario 4: A permanent shock

rbc_det4.mod

- Economy starts from the initial steady state ($a_0 = 0$)
- Period 1: TFP permanently increases by 5% (and this was unexpected)

```
...
initval;
    epsilon = 0;
end;

steady;

endval;
    epsilon = (1-rho)*log(1.05);
end;

steady;
```

Scenario 5: A pre-announced permanent shock

rbc_det5.mod

- Economy starts from initial steady state ($a_0 = 0$)
- Period 6: TFP permanently increases by 5%
- A shocks; block is used to maintain TFP at initial level during periods 1–5

```
...  
// Same initval and endval blocks as in Scenario 4  
...  
  
shocks;  
  var epsilon;  
  periods 1:5;  
  values 0;  
end;
```

Summary of commands

1. Only `initval` or `endval` followed by `steady`: set initial and terminal steady state
2. `histval` for initial state variable conditions out of steady state
→ `initval` followed by `steady` for terminal steady state
3. `initval` for initial condition (potentially followed by `steady`) and `endval` followed by `steady` for the terminal steady state
 - `shocks`; block for shocks along the simulation path
 - `perfect_foresight_setup` prepares the simulation
 - `perfect_foresight_solver` computes the simulation

Under the hood

- Paths for exogenous and endogenous variables are stored in two MATLAB/Octave matrices:

$$\begin{aligned} \text{oo_endo_simul} &= \begin{pmatrix} y_0 & y_1 & \dots & y_T & y_{T+1} \end{pmatrix} \\ \text{oo_exo_simul}' &= \begin{pmatrix} \boxtimes & u_1 & \dots & u_T & \boxtimes \end{pmatrix} \end{aligned}$$

- for historical reasons, dates are in columns in `oo_.endo_simul` and in lines in `oo_.exo_simul`, hence the transpose (')
- If there are exogenous leads/lags, u_0 and/or u_{T+1} would be in `oo_.exo_simul'` as well
→ more leads/lags would show up as auxiliary variables
- `perfect_foresight_setup` initializes those matrices, given the shocks, `initval`, `endval`, and `histval` blocks
 - y_0, y_{T+1} and $u_1 \dots u_T$ are the constraints of the problem
 - $y_1 \dots y_T$ are the initial guess for the Newton algorithm
- `perfect_foresight_solver` replaces $y_1 \dots y_T$ in `oo_.endo_simul` by the solution

Initial guess

- The Newton algorithm needs an initial guess $Y^{(0)} = [y_1^{(0)'} \dots y_T^{(0)'}]$.
- Dynare uses content of `oo_.endo_simul` created by `perfect_foresight_setup`
 - without `endval`, it is steady state as specified by `initval` (repeated for all simulations periods)
 - with `endval`, it is the terminal steady state declared within this block
- Opens possibility of manipulating `oo_.endo_simul` after `perfect_foresight_setup` (but of course before `perfect_foresight_solver!`)
- If `homotopy` is triggered, initial guess of subsequent iterations is result of previous iteration

Alternative way of specifying terminal conditions

- What if
 - final steady state is hard to compute?
 - model is very persistent and takes a long time to go back to steady state?
- `differentiate_forward_vars` option of `model` block allows computing terminal condition endogenously and avoiding final kink if T is not large enough
- This is achieved by substituting for forward-looking variables using new auxiliary variables:
 - Substitution: $x_{t+1} \rightarrow x_t + a_{t+1}$
 - New equation: $a_t = x_{t+1} - x_t$
- If the terminal condition is a steady state, the new auxiliary variables have obvious *zero* terminal condition
- Since Dynare 6: `endval_steady` option to `perfect_foresight_setup` to solve terminal steady state together with the dynamics

Outline

1. Presentation of the problem
2. Solution techniques
3. Shocks: temporary/permanent, unexpected/pre-announced
- 4. Recent Improvements**
5. Perfect foresight with expectation errors
6. Extended path

Dates as an alternative to period indices (1/2)

rbc_det_dates.mod

```
shocks;
  var epsilon;
  periods 2025Q3:2025Q4;
  values 0.2;
end;

perfect_foresight_setup(first_simulation_period = 2025Q2,
                        last_simulation_period = 2099Q4);
perfect_foresight_solver;

figure
plot(Simulated_time_series{'c'})
```

Dates as an alternative to period indices (2/2)

rbc_det_dates.mod

- Simulation available as a dseries object in variable Simulated_time_series

```
>> Simulated_time_series
```

```
Simulated_time_series is a dseries object:
```

	k	y	L	c	A	a	epsilon
2025Q1	19.2817	1.6493	0.31956	1.2637	1	0	0
2025Q2	19.1838	1.601	0.30525	1.3133	1	0	0
2025Q3	19.616	2.2002	0.36729	1.3843	1.2214	0.2	0.2
2025Q4	20.5823	2.8211	0.41631	1.4625	1.4333	0.36	0.2

- Still possible to use the periods option
- Dates also work with perfect_foresight_with_expectation_errors_solver (and the learnt_in option of the shocks block)

Controlled paths (1/2)

a.k.a. conditional forecast

```
var c k;  
varexo x;  
perfect_foresight_controlled_paths;  
    exogenize c;  
    periods 2 4:5;  
    values 1.6 1.7;  
    endogenize x;  
  
    exogenize k;  
    periods 7:9;  
    values 13;  
    endogenize x;  
end;
```

Controlled paths (2/2)

- Can be combined with regular `shocks` or `mshocks` blocks
- Compatible with regular perfect foresight solver and with expectations errors
- Compatible with most solvers (LU on stacked system, LBJ, iterative solvers. . .) and homotopy
- Not compatible with block decomposition and bytecode representation
- Supersedes the now deprecated `det_cond_forecast` command

Performance improvements for iterative solvers

- The core of the perfect foresight solver is the solution of a large sparse linear system

$$Ax = b \quad (40)$$

- Dynare provides two *iterative* solvers for that purpose that can be used with block decomposition:
 1. Generalized Minimal Residual (GMRES), under `stack_solve_algo=2`
 2. Biconjugate gradient stabilized method (BiCGStab), under `stack_solve_algo=3`
- These algorithms are much faster when A is “close” to the identity matrix
→ can improve performance by transforming the problem (“preconditioning”)

Preconditioning

- Central idea: find suitable matrix

$$M = M_1 M_2, \quad (41)$$

then proceed in two steps:

1. First, solve the equivalent system

$$\tilde{A}y \equiv (M_1^{-1} A M_2^{-1})(M_2 x) = M_1^{-1} b \equiv \tilde{b} \quad (42)$$

where $\tilde{A}$ is “closer” to identity than A

2. Second, recover the solution of the original system

$$x = M_2^{-1} y \quad (43)$$

- Shows tradeoff to finding good preconditioner:
 - M should be “reasonably close” to A
 - ideal case: inverse of A , such that $\tilde{A} = I$, (requires solving the original problem)
 - M_1 and M_2 are “easy” to invert/compute (e.g. triangular)
 - need to be applied in each iteration of the iterative solver

preconditioner-options in Dynare 7

- historical default: incomplete LU decomposition (`preconditioner=incomplete_lu`)
→ M_1, M_2 “close” to the LU decomposition of A
- `ilu` function from MATLAB/Octave is rather slow
- In Dynare 7 : two new preconditioners inspired from TROLL
 1. “First iteration LU” preconditioner (`preconditioner=first_iter_lu`; new default)
 - At first iteration of the *nonlinear* Newton solver, do a full LU decomposition (thus skip the iterative linear solver)
 - At further Newton iterations, use that decomposition as preconditioner
 2. “Block diagonal LU” preconditioner (`preconditioner=block_diagonal_lu`)
 - Compute full LU decomposition of a few simulation periods in middle of the stacked system
 - Repeat that small LU over block diagonal to create preconditioner for full stacked system
- More options to control the details of iterative solvers and block diagonal LU preconditioner

Performance improvements for iterative solvers

- Benchmark on GIMF with 4 countries, 100 periods, temporary demand shock, block decomposition, MATLAB R2024b
- LU on stacked system (`stack_solve_algo=0`, default): 34 s
- Iterative solver with `preconditioner=incomplete_lu` (old default)
 - GMRES (`stack_solve_algo=2`): 116 s
 - BiCGStab (`stack_solve_algo=3`): 115 s
- Iterative solver with `preconditioner=first_iter_lu` (new default)
 - GMRES (`stack_solve_algo=2`): 19 s
 - BiCGStab (`stack_solve_algo=3`): 19 s
- Iterative solver with `preconditioner=block_diagonal_lu`
 - GMRES (`stack_solve_algo=2`): 16 s
 - BiCGStab (`stack_solve_algo=3`): 17 s

New sparse linear solvers

- ParU: multifrontal multithreaded sparse LU factorization
→ promising for large models on machines with many CPU cores
- PARDISO: commercial/proprietary sparse linear solver (see panua.ch)
Two solver flavours:
 1. “Plain” PARDISO solver at every Newton iteration
 2. CGS+PARDISO: run PARDISO solver at first Newton iteration, then reuse the LU as preconditioner for iterative CGS solver in further Newton iterations
(similar to `preconditioner=first_iter_lu`)Seems the fastest available option on large models, especially the CGS+PARDISO flavour
- Solvers not yet in the unstable snapshot; will most likely be in Dynare 7

Outline

1. Presentation of the problem
2. Solution techniques
3. Shocks: temporary/permanent, unexpected/pre-announced
4. Recent Improvements
5. Perfect foresight with expectation errors
6. Extended path

Simulating unexpected shocks: perfect foresight with expectation errors

- With standard perfect foresight solver:
 - shocks are unexpected in period 1
 - but fully anticipated in subsequent periods
- How to simulate an unexpected shock at a period $t > 1$?
 1. Run perfect foresight simulation from 0 to T *without the shock*
 2. Run second perfect foresight simulation from periods t to T
 - applying the shock in t ,
 - and using the $t - 1$ results of the first simulation as initial condition
 3. Combine the two simulations:
 - use the first one for periods 1 to $t - 1$,
 - and the second one for t to T

An example with Dynare 6

rbc_expectation_errors.mod

- Simulation of a scenario with:
 - Pre-announced (negative) shocks in periods 5 and 15
 - Unexpected (positive) shock in period 10

```
shocks(learnt_in=1); // pre-announced shocks
    var epsilon;
    periods 5, 15;
    values -0.1, -0.1;
end;

shocks(learnt_in=10); // shocks learnt in period 10
    var epsilon;
    periods 10;
    values 0.1;
end;
```

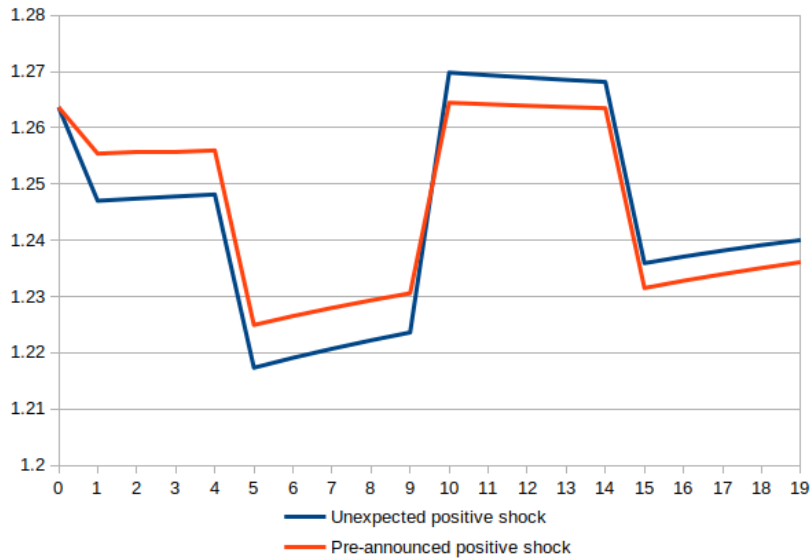

An example with Dynare 6

- Then the full simulation can be run with:

```
perfect_foresight_with_expectation_errors_setup(periods=300);  
perfect_foresight_with_expectation_errors_solver;
```

- More complex scenarios possible where agents learn in period $t > 1$ about shock(s) in periods $t + s$ for $s > 0$ (i.e. pre-announced shocks, but which are learnt in some period > 1)
- Information can also be learnt about terminal conditions, using `endval(learnt_in=...)` block

Consumption path comparison



Outline

1. Presentation of the problem
2. Solution techniques
3. Shocks: temporary/permanent, unexpected/pre-announced
4. Recent Improvements
5. Perfect foresight with expectation errors
6. Extended path

Extended path (EP) algorithm: idea

- Idea: use the previous method to simulate a *rational expectations* (RE) model (*i.e.* with stochastic shocks that can happen at every period)...
- ... but under the simplifying assumption that agents believe economy will not be perturbed in the future (*i.e.* future shocks are at steady state $u = 0$), and do not update their belief when observing shocks
→ solve an approximation to the true RE model
- Advantage: *deterministic* nonlinearities fully taken into account
- Inconvenient: solution under certainty equivalence violates Jensen's inequality
→ e.g. no precautionary motive.
- If goal is to generate a time series, method is superior to first-order perturbation (which is also certainty equivalent but ignores deterministic nonlinearities)
- Method introduced by Fair and Taylor (1983)

Extended path (EP) algorithm

- Algorithm

1. $H \leftarrow$ Set the horizon of the perfect foresight (PF) model
 2. $(y, u) \leftarrow$ Compute steady state of the model
 3. $y_0 \leftarrow$ Choose an initial condition for the endogenous variables
 4. for $t = 1$ to T
 5. $u_t \leftarrow$ Draw random shocks for the current period
 6. $y_t \leftarrow$ Solve a PF with:
 - Initial condition: y_{t-1} computed in previous iteration
 - Terminal condition: $y_{t+H} = y$
 - Shocks: u_t just drawn and $u_{t+s} = u$ (for $s > 0$)
 7. end for
- Implemented under the command `extended_path` (with option `order=0`, which is the default)
 - Option `periods` controls T
 - Option `solver_periods` controls H (defaults to 200)

Extended path: Dynare example

rbc_ep.mod

```
...  
// Declare shocks as in a stochastic setup  
shocks;  
    var epsilon;  
    stderr 0.02;  
end;  
  
extended_path(periods=300);  
  
// Plot 20 first periods of consumption  
ic = varlist_indices('c', M_.endo_names);  
plot(oo_.endo_simul(ic, 1:21));
```

Stochastic extended path (SEP)

- Idea: generalize the extended path method to take into account *some* future uncertainty
- Approximate rational expectations problem by one where agents at time t ,
 - know that there will be stochastic shocks in periods $t + 1$ to $t + k$
 - but assume no more shocks in periods after $t + k$
- k measures degree of future uncertainty taken into account
- The larger k , the closer we get to the true RE model (which corresponds to $k = +\infty$)
- Note: (plain) extended path (order=0) corresponds to $k = 0$
- Additional approximation: probability distribution about future uncertainty is simplified using discrete numerical integration (*a.k.a.* quadrature)

Gauss-Hermite quadrature (univariate)

- Let $X \sim N(0, \sigma_x^2)$ and suppose we need to evaluate $\mathbb{E}[\varphi(X)]$, where φ is a continuous function
- By definition we have:

$$\mathbb{E}[\varphi(X)] = \frac{1}{\sigma_x \sqrt{2\pi}} \int_{-\infty}^{\infty} \varphi(x) e^{-\frac{x^2}{2\sigma_x^2}} dx \quad (44)$$

- Integral can be approximated by a finite sum using Gauss-Hermite quadrature formula at order n :

$$\int_{-\infty}^{\infty} \varphi(z) e^{-z^2} dz \approx \sum_{i=1}^n \omega_i \varphi(z_i), \quad (45)$$

where z_i ($i = 1, \dots, n$) are the roots of an n th order Hermite polynomial, and the weights $\omega_i > 0$ sum up to one (variable change: $x_i = \frac{z_i}{\sigma_x \sqrt{2}}$)

- The higher n , the better the approximation

Gauss-Hermite quadrature (multivariate)

- Let X be a p -dimensional multivariate Gaussian random variable with mean zero and unit variance, and suppose we need to evaluate

$$\mathbb{E}[\varphi(X)] = (2\pi)^{-\frac{p}{2}} \int_{\mathbb{R}^p} \varphi(\mathbf{x}) e^{-\frac{1}{2}\mathbf{x}'\mathbf{x}} d\mathbf{x} \quad (46)$$

- Let $\{(\omega_i, z_i)\}_{i=1}^n$ be the weights and nodes of an order n univariate Gauss-Hermite quadrature
- This integral can be approximated using a tensor grid:

$$\int_{\mathbb{R}^p} \varphi(\mathbf{z}) e^{-\mathbf{z}'\mathbf{z}} d\mathbf{z} \approx \sum_{i_1, \dots, i_p=1}^n \omega_{i_1} \dots \omega_{i_p} \varphi(z_{i_1}, \dots, z_{i_p}) \quad (47)$$

- Curse of dimensionality 1:** number of terms in sum grows exponentially with number of shocks

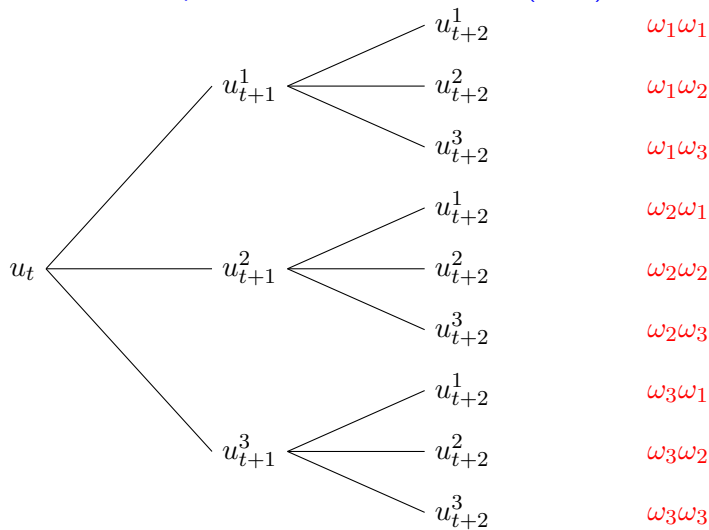
Other numerical integration rules

- Default: use multivariate Gaussian quadrature to approximate the expected terms
- To cope with **curse of dimensionality 1**, can consider other numerical integration approaches:
 - Cubature (order 3 or 5)
Set `options_.ep.IntegrationAlgorithm` to 'Stroud-Cubature-3'
or 'Stroud-Cubature-5'
 - Unscented transform
Set `options_.ep.IntegrationAlgorithm` to 'Unscented'

→ rely on smaller number of nodes which does not grow exponentially with number of innovations
- Unfortunately, there is another curse of dimensionality

Forward history

One shock, three quadrature nodes, order two SEP ($k = 2$)

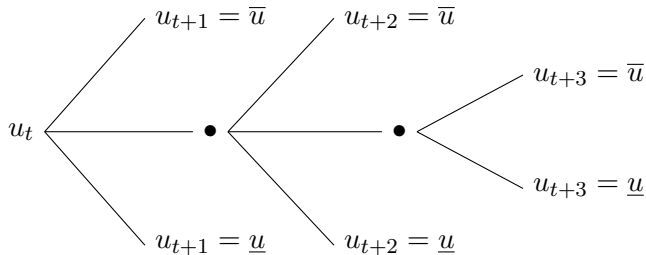


→ The tree of histories grows

Fishbone integration

- We can prune the tree of forward histories by removing low probability branches...
- Or by considering that innovations, say, at time $t + 1$ and $t + 2$ are unrelated variables
- Analogy with a good at different periods in an intertemporal general equilibrium model.
- If we have n_u innovations and if agents perceive uncertainty for the following k periods, we consider integration problem involving $n_u \times k$ unrelated variables
- Use two points Cubature rule to compute the integral (unscented transform with $\kappa = 0$)
→ complexity of integration problem grows linearly with n_u or k

Fishbone history (one shock, two nodes, order three SEP)



Stochastic extended path algorithm

Algorithm 1 Stochastic Extended path algorithm

1. $H \leftarrow$ Set the horizon of the *stochastic* perfect foresight (SPF) models.
 2. $(x^*, y^*) \leftarrow$ Compute steady state of the model.
 3. $\{(\omega_i, \omega_i)\}_{i=1}^m \leftarrow$ Get weights and nodes for numerical integration
 4. $(s_0, x_1) \leftarrow$ Choose an initial condition for the state variables
 5. **for** $t = 1$ to T **do**
 6. $u_t \leftarrow$ Draw random shocks for the current period
 7. $(y_t, x_{t+1}, s_t) \leftarrow$ Solve a SPF model with $y_{t+H+1} = y^*$
 8. **end for**
-

Stochastic extended path (SEP) (continued)

- SEP algorithm is similar to EP, but instead of solving a perfect foresight (PF) problem for each period, a larger problem is solved:
 - equations determining future variables are replicated as many times as there are branches on the tree of history;
 - Gauss-Hermite quadrature is used to compute expectations over those future variables
- We face two curses of dimensionality (exponential complexity growth):
 - Stochastic order (k)
 - Number of shocks
- In practice, only feasible for small k and small number of shocks
- SEP triggered with option `order=k` of `extended_path` command
- NB: currently no interface for controlling the number of nodes for the Gauss-Hermite quadrature
→ has to be directly set in the `options_` structure